

Lab8 矩阵乘加速器设计报告

2400011041 徐子翔

2026 年 6 月 30 日

1 设计目标

本设计实现 512×512 的有符号 8 bit 矩阵乘，并输出有符号 32 bit 结果。最终版本以功能正确、低功耗和低延迟为主要目标，并保持单时钟设计。

2 总体结构

提交包采用可运行目录结构，主要文件如下：

- `rtl/accelerator_top.v`: 功能仿真顶层，包含 controller、行为级 B SRAM 和计算核实例。
- `rtl/accelerator_logic_top.v`: logic-only PPA 综合顶层，只包含计算阵列及其必要门控逻辑。
- `rtl/accelerator_sram_ext.v`: PE/MAC 阵列。
- `rtl/sram_model.v`: pre-syn 功能仿真使用的同步 SRAM 行为模型。
- `testbench/pre-syn`: 完整功能仿真并生成 `result_mem.csv`。
- `syn/syn.tcl`: Design Compiler logic-only 综合脚本。

整体数据流为：controller 从 input memory 读取 A/B 矩阵数据，将当前 B column tile 写入两个 128 bit 行为 SRAM bank；A 矩阵采用 direct-A word buffer，不再额外实例化 A SRAM；计算核执行 1×32 tile 的 dot product 累加；结果经 result memory 接口按 64 bit word 写出。

3 提交包与运行方式

3.1 提交包结构

提交包根目录包含 RTL、testbench、综合脚本、输入数据、检查脚本、PDK 文件和本报告。主要入口如下：

- `Makefile`: 顶层运行入口。
- `input_mem.csv` 与 `in.npy`: 输入。
- `result_mem.csv`: pre-syn 的输出结果。
- `CheckResult.py`: 结果正确性检查脚本。
- `syn/log/area.rpt`、`syn/log/power.rpt`、`syn/log/timing.rpt`: 综合结果报告。
- `pdk/NanGate_15nm_OCL_slow_conditional_ccs.db`: Design Compiler 使用的 slow corner DB。
- `pdk/NanGate_15nm_OCL_conditional.v`: 门级仿真使用的标准单元 Verilog model。

3.2 运行方式

在支持 make、VCS 或 Icarus、Design Compiler 的环境中，可以在提交包根目录使用以下命令：

- `make`: 检查必要文件并运行 pre-syn 功能仿真，不依赖 Python 或 NumPy。
- `make all`: 运行 pre-syn 功能仿真、logic-only 综合和后综合 SDF 验证。
- `make generate_input`: 可选命令，用 Python 重新生成输入数据，需要 NumPy。
- `make check_result`: 可选命令，用 Python 检查 `result_mem.csv`，需要 NumPy。

综合脚本和 testbench 均优先支持完整 NanGate 目录结构；若不存在完整目录，则使用提交包中保留的 flat PDK 文件。因此当前提交包只需上述 DB 和 Verilog model 即可运行本设计流程。

4 关键设计参数

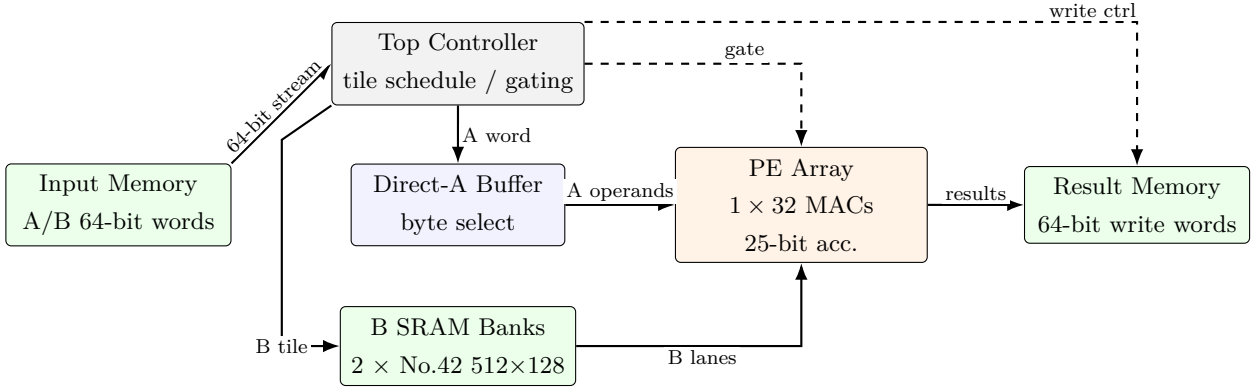


图 1: 最终加速器结构与数据流向

表 1: 最终 RTL 参数

参数	数值	说明
MATRIX_SIZE	512	矩阵维度
TILE_ROWS	1	每个 compute tile 的 A 行数
TILE_COLS	32	每个 compute tile 的 B 列数
B_BANKS	2	B SRAM bank 数量
ACC_BITS	25	PE 内部累加器位宽
结果输出位宽	32 bit	写回时符号扩展为 32 bit
SRAM macro	No.42 512x128	B tile 存储使用两个 bank
时钟约束	10.00 ns	单时钟, 100 MHz 功耗评估

选择 1×32 的原因是它在固定 PE 数较小的情况下减少 column tile 数, 并能充分使用两个 128 bit B SRAM bank。A 侧直接使用 64 bit input word buffer, 避免在 PPA 口径中引入额外 A SRAM macro。

5 主要优化方法

5.1 Direct-A 与 B tile 复用

A 矩阵按 64 bit word 读取, 每个 word 包含 8 个 SINT8 元素。controller 将 A word 保存在寄存器 buffer 中, 并逐 byte 提供给计算核。B 矩阵按 column tile 预先装入两个 512x128 SRAM bank, 之后在该 column tile 内跨所有 row tile 复用, 从而降低 B 载入开销。

5.2 A-zero 稀疏门控

输入数据具有较多零值。最终版本在 controller 中检测当前 A byte: 当 A byte 为 0 时, 不发出 PE pulse, 同时关闭对应 B SRAM read/prefetch 的 chip-select。logic-only 综合顶层中也使用 `pulse_en && (a_sram_rdata != 0)` 作为计算核使能, 使门控在综合 PPA 中可见。

5.3 PE 操作数隔离

PE 内部在 `pulse_en=0` 时将乘法器 A/B 输入置零, 仅在有效 pulse 时更新累加器。该结构降低无效周期的切换活动, 并保持累加结果不变。

5.4 计算地址气泡消除

行为级 SRAM 为同步读。早期版本每个 A word 计算前需要一个 `S_COMPUTE_ADDR` 读地址气泡。最终版本在 `S_LOAD_A_WRITE` 的最后一行同时发出第一个 B SRAM read, 使下一拍可以直接进入 `S_COMPUTE_PULSE`, 减少每个 A word 的空拍。

5.5 结果写回流式化

原始结果写回采用 setup/hold 两拍写一个 64 bit word。最终版本保留第一个 setup 拍, 之后在 `S_WRITE_HOLD` 中每拍写出上一个已稳定 word, 同时准备下一个 word 的地址和数据。最后一个 word 通过同步 result RAM 的下一拍写入完成 flush。该优化不改变 logic-only 计算核 PPA, 但将完整 pre-syn 周期数降至 5,464,066。

5.6 低漏电综合策略

综合脚本使用 logic-only 顶层 `accelerator_logic_top`, 时钟周期设为 10.00 ns, 并采用

```
compile_ultra -gate_clock -area_high_effort_script
```

该策略在满足 timing 的同时降低 cell area 和 leakage。综合脚本显式排除 SRAM macro、controller 和外部 memory, 符合 PPA 统计要求。

6 验证流程

在远端 clab 环境跑完整流程:

1. pre-syn VCS 跑功能仿真并生成 `result_mem.csv`;
2. 将 `result_mem.csv` 抓回本地, 在 Python venv 中运行 `CheckResult.py`;

3. Design Compiler 跑 logic-only 综合并生成 area/power/timing report、门级网表和 SDF。

验证输出如下：

- Pre-syn VCS: [TB] [LATENCY] cycles=5464066 realtime_ns=5464066.500
- Checker: Correct!, loss=0, relative_loss=0.0, sse=0.0

7 最终 PPA 结果

7.1 延迟

DC target clock 为 10.00 ns, 最差 setup slack 为 9,641.00 ps, 因此 shortest-period basis 为

$$T_{\text{short}} = 10000 \text{ ps} - 9641.00 \text{ ps} = 359.00 \text{ ps}. \quad (1)$$

完整功能仿真周期数为 5,464,066, 因此

$$L = 5,464,066 \times 0.359 \text{ ns} = 1,961,599.694 \text{ ns}. \quad (2)$$

7.2 逻辑综合结果

表 2: Design Compiler logic-only 结果

项目	数值
Total cell area	4296.081454 μm^2
Total dynamic power	0.3437407 mW
Cell leakage power	5.0590 mW
Logic total power	5.4027407 mW
Worst setup slack	9641.00 ps

7.3 SRAM 功耗统计

选用两个 No.42 512x128 SRAM macro。单个 macro 在 100 MHz 下 dynamic power 为 0.08043 mW, leakage 为 0.05155885 mW。A-zero 门控后, B SRAM active cycles 为

$$169,336 \times 16 + 16 \times 512 = 2,717,568. \quad (3)$$

总周期数为 5,464,066, 因此 B SRAM duty 为

$$\frac{2,717,568}{5,464,066} = 49.735270\%. \quad (4)$$

最终 sparse-aware SRAM power 为 0.183121856 mW。

7.4 总结果

最终指标为 SSE、latency 和 power 三项。SSE 由 checker 对输出矩阵和参考矩阵逐元素求平方误差得到：

$$\text{SSE} = \sum_{i,j} (C_{\text{out}}(i,j) - C_{\text{ref}}(i,j))^2 = 0. \quad (5)$$

Latency 使用功能仿真周期数和 DC timing 得到的 shortest-period basis 计算：

$$\text{Latency} = 5,464,066 \times (10,000 - 9,641) \text{ ps} = 1,961,599.694 \text{ ns}. \quad (6)$$

Power 采用 logic-only 综合功耗加 sparse-aware SRAM 功耗。逻辑功耗为

$$P_{\text{logic}} = 0.3437407 + 5.0590 = 5.4027407 \text{ mW}. \quad (7)$$

两个 B SRAM 的 duty 为 $2,717,568/5,464,066 = 49.735270\%$ ，因此

$$P_{\text{SRAM}} = 2 \times (0.05155885 + 0.08043 \times 0.49735270) = 0.183121856 \text{ mW}. \quad (8)$$

最终总功耗为

$$P_{\text{total}} = 5.4027407 + 0.183121856 = 5.585862556 \text{ mW} = 5,585,862,556 \text{ pW}. \quad (9)$$

表 3: 最终结果汇总

项目	数值
SSE	0
Latency	1,961,599.694 ns
Power	5,585,862,556 pW

8 结论

最终版本在保持 $\text{SSE} = 0$ 的前提下，采用 direct-A 1×32 计算结构、两个 512×128 B SRAM bank、A-zero 稀疏门控、PE 操作数隔离、计算地址气泡消除和结果写回流式化。完整远端验证通过，最终 latency 为 1,961,599.694 ns，总 sparse-aware power 为 5,585,862,556 pW。